Beyond Autocomplete: A Survey on the Evolution of Human-AI Pair Programming

Nikhil Anand

February 24, 2026

Introduction and Motivation

The integration of artificial intelligence into programming environments has transformed software development into a form of human–AI collaboration. Foundational research on recommender systems [1] and machine learning–based code quality assessment [2] established the basis for intelligent development tools, while recent large language models trained on code [3] have enabled AI systems to generate substantial functional code in real-world settings. Empirical studies report improvements in task completion time and developer satisfaction with AI assistance [4, 5], yet also highlight challenges related to trust calibration, verification effort, and overreliance [6]. Together, these findings suggest that AI-assisted programming represents a shift in the socio-cognitive dynamics of software engineering. This project aims to synthesize existing research to understand how human–AI collaboration influences productivity, cognition, trust, and code quality, and to identify open challenges for the design of human-centered AI development tools.

The course topics—including developer productivity measurement, cognitive processes in programming, socio-technical systems, and empirical research methods—provide a foundation for examining AI-assisted development. In AI pair programming contexts, traditional measures of productivity, cognitive load, and program comprehension must be reconsidered, as performance reflects joint human–AI contributions. This project will analyze how the emergence of AI pair programming reshapes core concepts in human-focused software engineering.

Learning Expectations

Through this project, I expect to gain a clearer understanding of how human–AI collaboration influences developer productivity, cognition, trust, and code quality. By synthesizing existing empirical research, I aim to identify key patterns, limitations, and open challenges in AI-assisted programming. This work will help clarify how human-centered software engineering principles must evolve in the era of AI pair programming.

Scope of the Survey

This survey will focus specifically on human–AI collaboration in programming environments, with particular emphasis on large language model–based code generation and AI pair programming systems deployed in professional development workflows. The review will examine empirical studies investigating productivity, trust calibration, cognitive load, verification behavior, and code quality outcomes. It will include both foundational work on intelligent development tools and recent large-scale studies of LLM-assisted programming. The survey will not address advances in model architecture, training techniques, or purely benchmark-driven code generation research, except where such work directly informs human-centered evaluation. The primary objective is to synthesize research that evaluates how AI systems influence developer behavior and decision-making in real-world programming contexts.

Work Plan

Week 1: Foundational literature review. Identify and review foundational research on intelligent development tools and early AI-assisted programming systems (approximately 5-7 papers) to establish historical context.

Week 2: LLM-era empirical studies. Survey recent LLM-era studies on AI pair programming, focusing on productivity, trust calibration, cognitive load, and code quality (5-3 additional papers).

Week 3: Comparative analysis. Conduct a comparative analysis of findings across studies, organizing the literature into thematic categories and identifying converging and conflicting results.

Week 4: Research question refinement. Refine and formalize research questions based on the synthesis, identify gaps in the literature, and incorporate additional targeted papers as needed.

Week 5: Writing and revision. Draft and revise the survey, integrating thematic insights and articulating open research challenges in human–AI collaboration in programming.

References

[1] Martin Robillard, Robert Walker, and Thomas Zimmermann. "Recommendation Systems for Software Engineering". In: IEEE Softw. 27.4 (July 2010), pp. 80–86. ISSN: 0740-7459. DOI: 10.1109/MS.2009.161. URL: https://doi.org/10.1109/MS.2009.161.

[2] Raymond P.L. Buse and Westley R. Weimer. "Learning a Metric for Code Readability". In: IEEE Transactions on Soft 36.4 (2010), pp. 546–558. DOI: 10.1109/TSE.2009.70.

[3] Mark Chen et al. Evaluating Large Language Models Trained on Code. 2021. arXiv: 2107.03374 [cs.LG]. URL: https://arxiv.org/abs/2107.03374.

[4] Albert Ziegler et al. "Productivity assessment of neural code completion". In: Proceedings of the 6th ACM SIGPLAN MAPS 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, pp. 21–29. ISBN: 9781450392730. DOI: 10.1145/3520312.3534864. URL: https://doi.org/10.1145/3520312.3534864.

[5] Sida Peng et al. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. 2023. arXiv: 2302.06590 [cs.SE]. URL: https://arxiv.org/abs/2302.06590.

[6] Priyan Vaithilingam et al. "Expectations, Outcomes, and Challenges of Modern Code Completion Tools". In: CHI Conference on Human Factors in Computing Systems. 2022.